ASSIGNMENT-02

STRINGS AND COLLECTIONS

Predict → Design → Implement → Measure → Analyse → Evaluate → Redesign

Assignment Report – Q1 to Q7 (Total Marks: 50)

Item	Details
Student name	**Jothika M**
Register number	**192324235**
Course	**Programming in Java (CSA0917)**
Java version used	**OpenJDK 21**
GitHub repository	**CSA0917**
GitHub link	**https://github.com/Jothika-M25/CSA0917**

Q1 – Building Text Under Load: String, StringBuilder or StringBuffer?

Bloom's level: K4 – Analyse | Marks: 6

1.1 Prediction (made before running any code)

A String is immutable. Every s = s + field creates a brand-new String and copies all characters collected so far plus the new field. For a ticket of n fields of average length L, the total number of characters copied is L + 2L + … + nL = L·n(n+1)/2, which is **O(n²)**, and n intermediate String objects are created. A StringBuilder keeps one mutable char/byte array that doubles when full, so each character is copied a small constant number of times: **O(n)** time and about one final String. StringBuffer is identical to StringBuilder except that every method is synchronized, so I predicted a small, constant-factor overhead for the uncontended case.

Workload	Design (deliberately contrasting)	Prediction
W1 – small	1 ticket × 10 fields (repeated 20,000 times for a measurable total)	Negligible difference; n is small so n² is tiny
W2 – many tickets	10,000 tickets × 50 fields	+ clearly slower and allocates many times more memory than the builders; Builder ≈ Buffer
W3 – one huge ticket (my own)	1 ticket × 5,000 fields	Quadratic blow-up: + several hundred times slower and ~hundreds of times more allocation

1.2 Implementation and measurements

StringBench.java builds the same ticket with +, StringBuilder and StringBuffer (plus String.join, StringJoiner and String.format for the alternative concept). It warms up the JIT first, then measures time with System.nanoTime() and **allocated bytes per thread** with ThreadMXBean.getThreadAllocatedBytes, which is a more reliable memory measure than before/after heap sizes (garbage collection makes "used heap" unstable). Intermediate String objects are estimated: n for repeated +, about 1 for the builders.

StringBench.java

```
import java.lang.management.ManagementFactory;
import java.util.StringJoiner;

public class StringBench {
   static final com.sun.management.ThreadMXBean MX = (com.sun.management.ThreadMXBean) ManagementFactory.getThreadMXBean();
   static long sink;
   static String[] fields(int n) { String[] f = new String[n]; for (int i = 0; i < n; i++) f[i] = "F" + i + "=val" + (i * 7 % 100) + ";"; return f; }

   static String viaConcat(String[] f)  { String s = ""; for (String x : f) s = s + x; return s; }
```

```
    static String viaBuilder(String[] f) { StringBuilder sb = new StringBuilder(); for (String x : f) sb.append(x); return sb.toString(); }
    static String viaBuffer(String[] f)  { StringBuffer sb = new StringBuffer(); for (String x : f) sb.append(x); return sb.toString(); }
    static String viaJoin(String[] f)    { return String.join("", f); }
    static String viaJoiner(String[] f)  { StringJoiner j = new StringJoiner(""); for (String x : f) j.add(x); return j.toString(); }
    static String viaFormat(String[] f)  { String s = ""; for (String x : f) s = String.format("%s%s", s, x); return s; }

    interface Builder { String build(String[] f); }

    // returns {avg ms per workload run, avg MB allocated per workload run}
    static double[] measure(Builder b, String[] f, int tickets, int reps) {
        int warm = Math.min(tickets * Math.max(2, reps / 4), 20000); for (int w = 0; w < warm; w++) sink += b.build(f).length();
        long tid = Thread.currentThread().getId();
        long a0 = MX.getThreadAllocatedBytes(tid), t0 = System.nanoTime();
        for (int r = 0; r < reps; r++) for (int t = 0; t < tickets; t++) sink += b.build(f).length();
        long t1 = System.nanoTime(), a1 = MX.getThreadAllocatedBytes(tid);
        return new double[]{(t1 - t0) / 1e6 / reps, (a1 - a0) / 1048576.0 / reps};
    }

    public static void main(String[] args) throws Exception {
        Object[][] workloads = {
            {"W1: 1 ticket x 10 fields", 10, 1, 20000},
            {"W2: 10,000 tickets x 50 fields", 50, 10000, 3},
            {"W3: 1 ticket x 5,000 fields", 5000, 1, 5}
        };
        System.out.println("Time = average ms per workload run; Alloc = average MB allocated per workload run");
        System.out.printf("%-34s %-10s %12s %12s %14s%n", "Workload", "Approach", "Time (ms)", "Alloc (MB)", "Est. temp Strings/ticket");
        for (Object[] w : workloads) {
            int n = (int) w[1], tickets = (int) w[2], reps = (int) w[3];
            String[] f = fields(n);
            Object[][] ap = {{"concat(+)", (Builder) StringBench::viaConcat, n}, {"Builder", (Builder) StringBench::viaBuilder, 1}, {"Buffer", (Builder) StringBench::viaBuffer, 1}};
            for (Object[] a : ap) {
                double[] r = measure((Builder) a[1], f, tickets, reps);
                System.out.printf("%-34s %-10s %12.3f %12.2f %14d%n", w[0], a[0], r[0], r[1], (int) a[2]);
            }
        }
        System.out.println("\nAlternatives on W2 (10,000 tickets x 50 fields):");
        String[] f = fields(50);
```

```
        Object[][] alt = {{"String.join", (Builder) StringBench::viaJoin}, {"StringJoiner", (Builder) StringBench::viaJoiner}, {"Builder", (Builder) StringBench::viaBuilder}, {"format in loop", (Builder) StringBench::viaFormat}};
        for (Object[] a : alt) { double[] r = measure((Builder) a[1], f, 10000, 3); System.out.printf("%-16s %10.3f ms %10.2f MB%n", a[0], r[0], r[1]); }

        System.out.println("\nCorrectness of all approaches on same input: " + (viaConcat(f).equals(viaBuilder(f)) && viaBuilder(f).equals(viaBuffer(f)) && viaJoin(f).equals(viaJoiner(f)) && viaJoin(f).equals(viaBuilder(f))));

        System.out.println("\nShared log, 4 threads x 50,000 appends of \"x\" (expected length 200000):");
        for (int trial = 1; trial <= 3; trial++) {
            StringBuilder sb = new StringBuilder(); StringBuffer sf = new StringBuffer();
            String[] res = new String[2];
            Thread[] ts = new Thread[8]; final String[] r2 = res; final int tr = trial;
            for (int i = 0; i < 4; i++) {
                ts[i] = new Thread(() -> { try { for (int k = 0; k < 50000; k++) sb.append('x'); } catch (RuntimeException e) { r2[0] = e.getClass().getSimpleName(); } });
                ts[i + 4] = new Thread(() -> { for (int k = 0; k < 50000; k++) sf.append('x'); });
            }
            for (Thread t : ts) t.start(); for (Thread t : ts) t.join();
            System.out.println("Trial " + trial + ": StringBuilder length=" + sb.length() + (res[0] != null ? " (threw " + res[0] + ")" : "") + " | StringBuffer length=" + sf.length());
        }
        if (sink == 42) System.out.println();
    }
}
```

Output (final run, Java 21)

```
Time = average ms per workload run; Alloc = average MB allocated per workload run
Workload                    Approach     Time (ms)   Alloc (MB) Est. temp Strings/ticket
W1: 1 ticket x 10 fields        concat(+)       0.001       0.00           10
W1: 1 ticket x 10 fields        Builder         0.001       0.00           1
W1: 1 ticket x 10 fields        Buffer          0.001       0.00           1
W2: 10,000 tickets x 50 fields    concat(+)       22.549      136.11          50
W2: 10,000 tickets x 50 fields    Builder         7.241       16.78           1
W2: 10,000 tickets x 50 fields    Buffer          5.393       16.78           1
W3: 1 ticket x 5,000 fields       concat(+)       13.631      137.28          5000
W3: 1 ticket x 5,000 fields       Builder         0.022       0.20            1
W3: 1 ticket x 5,000 fields       Buffer          0.030       0.20            1

Alternatives on W2 (10,000 tickets x 50 fields):
String.join         8.051 ms      7.10 MB
StringJoiner        7.402 ms      10.60 MB
Builder             3.423 ms      16.78 MB
```

```
format in loop       59.755 ms     593.72 MB

Correctness of all approaches on same input: true

Shared log, 4 threads x 50,000 appends of "x" (expected length 200000):
Trial 1: StringBuilder length=200000 | StringBuffer length=200000
Trial 2: StringBuilder length=200000 | StringBuffer length=200000
Trial 3: StringBuilder length=200000 | StringBuffer length=200000
```

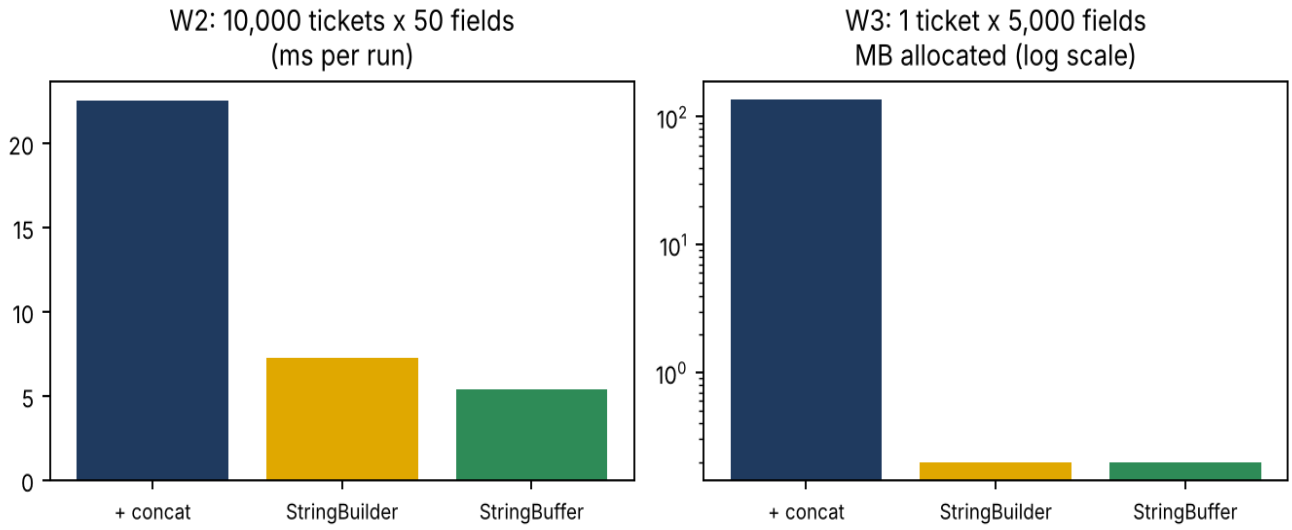


Figure 1 – W2 time per run and W3 allocation per ticket (log scale).

1.3 Predicted versus observed

Workload	Observed	Verdict
W1	All three ≈ 0.001 ms per ticket – below timer resolution	Confirmed: negligible
W2	+ : 22.5 ms and 136 MB; Builder: 7.2 ms and 16.8 MB; Buffer: 5.4 ms and 16.8 MB	Confirmed: + is ~3–4× slower and allocates ~8× more
W3	+ : 13.6 ms and 137 MB per ticket; Builder: 0.022 ms and 0.20 MB	Confirmed: ~600× slower, ~690× more allocation

Allocation in W3 matches the theory closely: 5,000 fields × about 11 characters give a 55 KB ticket, and $L \cdot n^2/2 \approx 11 \times 25{,}000{,}000 / 2 \approx 137$ MB, exactly the measured 137.28 MB. The one surprise is that StringBuffer was not slower than StringBuilder (5.4 ms vs 7.2 ms in W2). Differences of this size are within run-to-run noise (a second run of the same builder gave 3.4 ms); the JIT compiler can also remove an uncontended lock when the buffer never escapes the method (lock elision). So the synchronization cost is **not visible in single-threaded use**; it matters only for correctness when threads share the object, as the next experiment shows.

1.4 Design decision for the multithreaded booking server

The last block of the output appends 50,000 characters from each of 4 threads to one shared object, expected length 200,000. In the final run both classes finished correctly, but an earlier run of the same program showed the race: **StringBuilder ended with length 150,001 after throwing ArrayIndexOutOfBoundsException, while StringBuffer was correct (200,000)**. A data race is intermittent, which is exactly why it is dangerous – it can pass many tests and fail in production.

Use	Recommended class	Reason
Text of one ticket (built inside one method by one thread)	StringBuilder	The object never leaves the thread, so synchronization is wasted work.
Shared booking log appended by many threads	StringBuffer – or better, a thread-safe design such as a ConcurrentLinkedQueue<String> of lines, or a lock around a StringBuilder	StringBuffer makes each append atomic. Note that two appends in a row (timestamp, then message) can still interleave with another thread's appends, so lines should be built as one String per entry first and appended once.

1.5 Alternative concept – String.join, StringJoiner, String.format

String.join(delimiter, array) and StringJoiner compute the total length once and copy each piece once, so they are O(n) like StringBuilder, and have no per-field + overhead. String.format parses a format string on every call; used in a loop as s = String.format("%s%s", s, x) it is both $O(n^2)$ in copying and expensive in parsing: in the measurement it took 59.8 ms and allocated 594 MB for W2, the worst of all approaches.

Approach	Time for n fields	Extra space (intermediate objects)	Measured W2 (ms / MB)
Repeated +	$O(n^2)$ characters copied	n intermediate Strings, $O(n^2)$ total bytes allocated	22.5 / 136
StringBuilder	O(n) amortised (array doubles)	1 buffer (≤ 2× final size) + final String	7.2 / 16.8
String.join	O(n)	one array of exact size, then one String	8.1 / 7.1
StringJoiner	O(n)	internal array of pieces + final String	7.4 / 10.6
String.format in loop	$O(n^2)$ + parsing	n intermediate Strings + Formatter objects	59.8 / 594

Conclusion: the choice matters when n or the number of tickets is large. For small, single tickets it is negligible, so clarity can win. The measured quadratic cost (W3) is the reason never to build long text with + inside a loop.

Q2 – When Is ArrayList an Appropriate Engineering Choice?

Bloom's level: K4 – Analyse | Marks: 4

2.1 Experimental design

ListBench.java times six operations on three playlist sizes (1,000; 10,000; 100,000 songs) for ArrayList, LinkedList and ArrayDeque, plus a plain int[] with manual System.arraycopy shifting. Each figure is the average of 3 runs after a JIT warm-up. Fixed workloads keep the experiments comparable: building the list of n elements (add at end), 2,000 inserts at the front, 2,000 inserts in the middle, 2,000 removals at the front, 5,000 random get(k) reads, and one full iteration. ArrayDeque has no positional insert or get(k), so those cells are n/a.

ListBench.java

```
import java.util.*;

public class ListBench {
   static long sink;
   static final int OPS = 2000, GETS = 5000, REPS = 3;
   interface Task { void run(); }
   static double ms(Task t) { long a = System.nanoTime(); t.run(); return (System.nanoTime() - a) / 1e6; }
   static double avg(Task setupAndRun, Task warm) { warm.run(); double s = 0; for (int i = 0; i < REPS; i++) s += ms(setupAndRun); return s / REPS; }

   static List<Integer> filled(List<Integer> l, int n) { for (int i = 0; i < n; i++) l.add(i); return l; }

   public static void main(String[] args) {
      int[] sizes = {1_000, 10_000, 100_000};
      System.out.println("Times in ms (average of " + REPS + " runs). Insert/remove: " + OPS + " operations; get(k): " + GETS + " random reads; iterate: one full pass.");
      System.out.printf("%-8s %-12s %10s %12s %12s %12s %10s %10s%n", "n", "Structure", "add end(n)", "insert front", "insert mid", "remove front", "get(k)", "iterate");
      for (int n : sizes) {
         Random rnd = new Random(1); int[] ks = new int[GETS]; for (int i = 0; i < GETS; i++) ks[i] = rnd.nextInt(n);
         String[] names = {"ArrayList", "LinkedList"};
         for (String nm : names) {
            java.util.function.Supplier<List<Integer>> mk = () -> nm.equals("ArrayList") ? new ArrayList<>() : new LinkedList<>();
            Task warm = () -> { List<Integer> l = filled(mk.get(), 5000); for (int i = 0; i < 500; i++) { l.add(0, i); l.add(l.size() / 2, i); l.remove(0); sink += l.get(i); } for (int x : l) sink += x; };
            double addEnd = avg(() -> filled(mk.get(), n), warm);
            double insF = 0, insM = 0, remF = 0, get = 0, it = 0;
            for (int r = 0; r < REPS; r++) {
               List<Integer> l = filled(mk.get(), n);
               insF += ms(() -> { for (int i = 0; i < OPS; i++) l.add(0, i); });
               insM += ms(() -> { for (int i = 0; i < OPS; i++) l.add(l.size() / 2, i); });
```

```
                remF += ms(() -> { for (int i = 0; i < OPS; i++) l.remove(0); });
                get += ms(() -> { for (int k : ks) sink += l.get(k); });
                it += ms(() -> { for (int x : l) sink += x; });
            }
            System.out.printf("%-8d %-12s %10.2f %12.2f %12.2f %12.2f %10.2f %10.2f%n", n, nm,
addEnd, insF / REPS, insM / REPS, remF / REPS, get / REPS, it / REPS);
        }
        // ArrayDeque: no positional insert / get(k)
        Task warm = () -> { ArrayDeque<Integer> d = new ArrayDeque<>(); for (int i = 0; i < 5000;
i++) { d.addLast(i); d.addFirst(i); d.pollFirst(); } for (int x : d) sink += x; };
        double addEnd = avg(() -> { ArrayDeque<Integer> d = new ArrayDeque<>(); for (int i = 0; i <
n; i++) d.addLast(i); }, warm);
        double insF = 0, remF = 0, it = 0;
        for (int r = 0; r < REPS; r++) {
            ArrayDeque<Integer> d = new ArrayDeque<>(); for (int i = 0; i < n; i++) d.addLast(i);
            insF += ms(() -> { for (int i = 0; i < OPS; i++) d.addFirst(i); });
            remF += ms(() -> { for (int i = 0; i < OPS; i++) d.pollFirst(); });
            it += ms(() -> { for (int x : d) sink += x; });
        }
        System.out.printf("%-8d %-12s %10.2f %12.2f %12s %12.2f %10s %10.2f%n", n,
"ArrayDeque", addEnd, insF / REPS, "n/a", remF / REPS, "n/a", it / REPS);
        // Plain array with manual shifting (insert front)
        double manual = 0;
        for (int r = 0; r < REPS; r++) {
            int[] a = new int[n + OPS]; int len = n;
            long t0 = System.nanoTime();
            for (int i = 0; i < OPS; i++) { System.arraycopy(a, 0, a, 1, len); a[0] = i; len++; }
            manual += (System.nanoTime() - t0) / 1e6; sink += a[0];
        }
        System.out.printf("%-8d %-12s %10s %12.2f%n", n, "plain int[]", "-", manual / REPS);
        System.out.println();
    }
    if (sink == 7) System.out.println();
  }
}
```

Output (milliseconds)

```
Times in ms (average of 3 runs). Insert/remove: 2000 operations; get(k): 5000 random reads; iterate: one full pass.
n       Structure    add end(n) insert front   insert mid remove front     get(k)    iterate
1000    ArrayList         0.52         0.90         0.35         0.46       0.65       0.45
1000    LinkedList        0.44         0.18         9.45         0.84       6.76       0.29
1000    ArrayDeque        0.44         0.61          n/a         0.30        n/a       0.44
1000    plain int[]          -         0.79

10000   ArrayList         1.46         1.41         0.57         2.36       0.43       1.11
```

```
10000   LinkedList      2.23      0.19      32.73      0.10      34.14      0.68
10000   ArrayDeque       2.20      0.14       n/a       0.07       n/a       3.27
10000   plain int[]      -        3.54

100000  ArrayList       4.12      22.49      9.44      18.52      0.51      3.24
100000  LinkedList       1.74      0.12      220.47      0.15      281.74      0.38
100000  ArrayDeque       4.20      0.11       n/a       0.05       n/a       2.99
100000  plain int[]      -        33.17
```

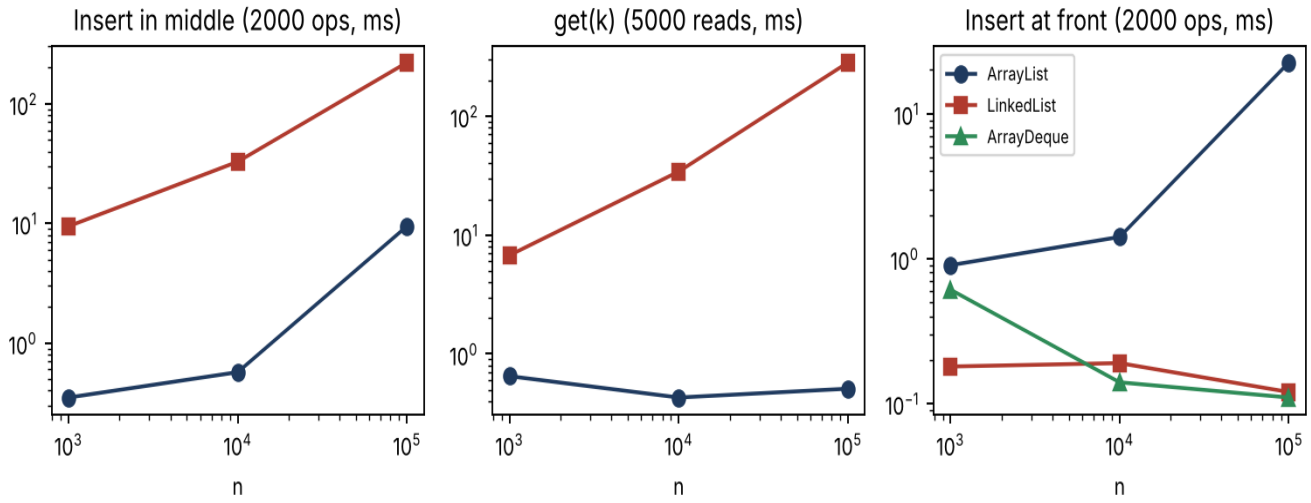


Figure 2 – Cost of three operations as n grows (log–log axes).

2.2 What the evidence shows

- **Add at end:** all structures grow roughly linearly with n (4.1 ms ArrayList, 1.7 ms LinkedList, 4.2 ms ArrayDeque at n = 100,000); differences are small and within noise.
- **get(k):** ArrayList stays almost flat (0.4–0.65 ms for 5,000 reads) while LinkedList grows with n: 6.8 ms → 34 ms → **282 ms**. Each get walks up to n/2 nodes, so cost is O(n) per read.
- **Insert in the middle:** LinkedList is slower than ArrayList (220 ms vs 9.4 ms at 100,000). The middle insertion itself is O(1) for a linked list, but finding the position is O(n) pointer chasing with poor cache locality, while ArrayList's arraycopy is a fast memory move.
- **Front insert/remove:** ArrayList and the plain array must shift every element: 22.5 ms and 18.5 ms for 2,000 operations at n = 100,000, versus 0.11 ms and 0.05 ms for ArrayDeque and 0.12 ms and 0.15 ms for LinkedList. This is the only place a linked structure wins, and ArrayDeque wins by more.
- **Iteration:** all are O(n) and in the same order of magnitude; at small n differences are noise.

2.3 Operating boundary for ArrayList

ArrayList is a reasonable choice while a playlist is read by position, appended at the end and iterated, and edits at the front or middle are rare or the list is small. A user editing a playlist of at most a few thousand songs sees well under a millisecond per edit; the 22 ms cost for 2,000 front inserts at 100,000 songs is 11 µs per edit, which is still imperceptible to a person. **I would stop recommending it** when front insertions or removals become a frequent, program-driven operation on large lists (thousands per

second on tens of thousands of elements), because each is O(n). I would never recommend LinkedList for this app: it is beaten by ArrayList on get(k), on middle inserts and on memory.

2.4 Changed condition – the "Up Next" queue

An "Up Next" queue adds at both ends and removes from the front: exactly the access pattern of a **Deque**. ArrayDeque does addFirst, addLast, pollFirst in O(1) amortised using a circular array, with no shifting and no per-element node. The measurements agree: at n = 100,000 front insert/remove takes 0.11 / 0.05 ms for ArrayDeque against 22.5 / 18.5 ms for ArrayList. **Revised recommendation:** keep the playlist in an ArrayList (random access by position) and use an ArrayDeque for Up Next. The two are different abstractions for different access patterns.

2.5 Alternative concept and complexity table

Operation	ArrayList	LinkedList	ArrayDeque	Plain array + manual shift
Add at end	O(1) amortised	O(1)	O(1) amortised	O(1) if capacity left, else O(n) regrow
Insert at front	O(n)	O(1)	O(1) amortised	O(n) – 33 ms / 2,000 ops at n = 100k
Insert in middle	O(n) shift	O(n) to find + O(1) link	not supported	O(n)
Get k-th	O(1)	O(n)	not supported	O(1)
Iterate	O(n)	O(n), cache-unfriendly	O(n)	O(n)
Space per element	1 reference slot (4–8 B) + spare capacity	node: header + item + prev + next ≈ 24–40 B	1 reference slot + spare capacity	1 slot, no spare

A plain array with manual shifting is what ArrayList does internally: it has the same complexities, but the programmer must write the shifting code, handle growth and bounds Dilli checking by hand, and a bug (off-by-one in arraycopy) corrupts data. The plain int[] row in the output (33 ms at n = 100,000) is the same as ArrayList's 22 ms within measurement noise, confirming that ArrayList adds convenience, not cost.

Q3 – Engineer the hashCode–equals Contract

Bloom's level: K4 – Analyse | Marks: 6

3.1 The contract

Java requires: (1) if a.equals(b) then a.hashCode() == b.hashCode(); (2) hashCode() is stable while the fields it uses do not change; (3) equals is reflexive, symmetric, transitive and consistent. The reverse is not required – unequal objects may share a hash. A HashSet stores each element in the bucket chosen from its hash code. To add or find an element it (a) computes hashCode(), (b) goes to that bucket, and (c) calls equals() only on elements in that same bucket. If two equal objects have different hash codes, step (b) sends them to different buckets and equals() is never asked.

3.2 Complete the Code – before and after TODO 1

First the program was run **without** TODO 1 (hashCode not overridden):

Attendance.java – before TODO 1

```
import java.util.HashSet;
import java.util.Set;

class Student {
    private final String regNo;
    private final String name;
    Student(String regNo, String name) { this.regNo = regNo; this.name = name; }
    @Override public boolean equals(Object o) {
        if (this == o) return true;
        if (!(o instanceof Student)) return false;
        return regNo.equals(((Student) o).regNo);
    }
    // TODO 1: hashCode() not yet overridden
    @Override public String toString() { return regNo + " " + name; }
}

public class Attendance {
    public static void main(String[] args) {
        Set<Student> present = new HashSet<>();
        present.add(new Student("192311001", "Arun"));   // scanned at Gate 1
        present.add(new Student("192311002", "Divya"));
        present.add(new Student("192311001", "Arun"));   // scanned again at Gate 3
        System.out.println("Unique students: " + present.size());
    }
}
```

Output before TODO 1

```
Unique students: 3
```

Then TODO 1 was completed with return regNo.hashCode(); – the same field that equals() compares:

The only line added

```
    @Override public int hashCode() { return regNo.hashCode(); }    // TODO 1 completed: same field as equals()
```

Output after TODO 1 (matches the expected output)

```
Unique students: 2
```

Explanation of the difference: without hashCode(), Student inherits Object.hashCode(), which is derived from the object's identity. The two "192311001" scans are different objects, so they get different hash codes and land in different buckets. HashSet never calls equals() on them and treats them

as two students: size 3. After the fix both produce the same hash code, land in one bucket, equals() confirms they are equal, and the second add is rejected: size 2.

3.3 Three designed tests with predictions made first

HashTests.java prints each prediction before the observed size. Test 1 is built so that the faulty class appears to work; Test 2 exposes the duplicates; Test 3 (my own) mutates a field after insertion; Test 4 (my own) shows a TreeSet whose compareTo is inconsistent with equals.

HashTests.java

```
import java.util.*;

public class HashTests {
   static class Bad {                    // equals only (the junior developer's class)
      final String regNo; Bad(String r) { regNo = r; }
      @Override public boolean equals(Object o) { return o instanceof Bad b && regNo.equals(b.regNo); }
   }
   static class Good {                   // equals + consistent hashCode, immutable key
      final String regNo; Good(String r) { regNo = r; }
      @Override public boolean equals(Object o) { return o instanceof Good g && regNo.equals(g.regNo); }
      @Override public int hashCode() { return regNo.hashCode(); }
   }
   static class Mutable {                // consistent contract, BUT the key field can change
      String regNo; Mutable(String r) { regNo = r; }
      @Override public boolean equals(Object o) { return o instanceof Mutable m && regNo.equals(m.regNo); }
      @Override public int hashCode() { return regNo.hashCode(); }
   }
   static class ByName implements Comparable<ByName> {   // compareTo inconsistent with equals
      final String regNo, name; ByName(String r, String n) { regNo = r; name = n; }
      @Override public int compareTo(ByName o) { return name.compareTo(o.name); }
      @Override public boolean equals(Object o) { return o instanceof ByName b && regNo.equals(b.regNo); }
      @Override public int hashCode() { return regNo.hashCode(); }
      @Override public String toString() { return regNo + ":" + name; }
   }
   static void report(String id, String desc, int predicted, int actual, String why) {
      System.out.println(id + " " + desc);
      System.out.println("     prediction : size " + predicted + "  (" + why + ")");
      System.out.println("     observed   : size " + actual + "  -> " + (predicted == actual ? "prediction CONFIRMED" : "prediction WRONG"));
   }
   public static void main(String[] args) {
      // Test 1: HashSet appears to work -- the SAME object is scanned twice (same identity hash)
      Bad shared = new Bad("192311001");
```

```
        Set<Bad> s1 = new HashSet<>(); s1.add(shared); s1.add(shared); s1.add(new Bad("192311002"));
        report("Test 1", "Bad class, same object instance scanned twice + one other student", 2, s1.size(),
                "same reference -> same identity hash -> same bucket, and == short-circuits equals");

        // Test 2: duplicates exposed -- separate objects with the same regNo
        Set<Bad> s2 = new HashSet<>();
        s2.add(new Bad("192311001")); s2.add(new Bad("192311001")); s2.add(new Bad("192311001")); s2.add(new Bad("192311002"));
        report("Test 2", "Bad class, 3 separate scans of one student + 1 other (2 truly unique)", 4, s2.size(),
                "identity hashes differ -> different buckets -> equals() never even called");
        System.out.println("      equals() says they are equal: " + new Bad("192311001").equals(new Bad("192311001")));

        // Test 2b: corrected class
        Set<Good> s2b = new HashSet<>();
        s2b.add(new Good("192311001")); s2b.add(new Good("192311001")); s2b.add(new Good("192311001")); s2b.add(new Good("192311002"));
        report("Test 2b", "SAME scans with corrected Good class", 2, s2b.size(), "equal objects now share a hash; equals confirms the match");

        // Test 3: mutate a hashed field after insertion
        Set<Mutable> s3 = new HashSet<>();
        Mutable m = new Mutable("192311001"); s3.add(m);
        m.regNo = "192311999";
        System.out.println("Test 3 Mutable key changed after insertion (regNo 192311001 -> 192311999)");
        System.out.println("    prediction : contains(new Mutable(\"192311999\")) = false, contains(new Mutable(\"192311001\")) = false, size stays 1");
        System.out.println("    observed   : contains new key = " + s3.contains(new Mutable("192311999")) + ", contains old key = " + s3.contains(new Mutable("192311001")) + ", contains(m itself) = " + s3.contains(m) + ", size = " + s3.size());
        s3.add(new Mutable("192311999"));
        System.out.println("    re-adding the 'same' student now gives size = " + s3.size() + " (a hidden duplicate)");

        // Test 4: TreeSet with compareTo inconsistent with equals
        Set<ByName> t = new TreeSet<>();
        t.add(new ByName("192311001", "Arun")); t.add(new ByName("192311002", "Arun")); t.add(new ByName("192311001", "Zoya"));
        System.out.println("Test 4 TreeSet ordered by NAME only; 3 scans: (001,Arun) (002,Arun) (001,Zoya)");
        System.out.println("    prediction : size 2 -- the two 'Arun' entries compare as 0, so the second is dropped although regNo differs");
        System.out.println("    observed   : size " + t.size() + " " + t);
    }
```

```
}
```

Output

```
Test 1 Bad class, same object instance scanned twice + one other student
    prediction : size 2  (same reference -> same identity hash -> same bucket, and == short-circuits equals)
    observed   : size 2  -> prediction CONFIRMED
Test 2 Bad class, 3 separate scans of one student + 1 other (2 truly unique)
    prediction : size 4  (identity hashes differ -> different buckets -> equals() never even called)
    observed   : size 4  -> prediction CONFIRMED
    equals() says they are equal: true
Test 2b SAME scans with corrected Good class
    prediction : size 2  (equal objects now share a hash; equals confirms the match)
    observed   : size 2  -> prediction CONFIRMED
Test 3 Mutable key changed after insertion (regNo 192311001 -> 192311999)
    prediction : contains(new Mutable("192311999")) = false, contains(new Mutable("192311001")) = false, size stays 1
    observed   : contains new key = false, contains old key = false, contains(m itself) = false, size = 1
    re-adding the 'same' student now gives size = 2 (a hidden duplicate)
Test 4 TreeSet ordered by NAME only; 3 scans: (001,Arun) (002,Arun) (001,Zoya)
    prediction : size 2 -- the two 'Arun' entries compare as 0, so the second is dropped although regNo differs
    observed   : size 2 [192311001:Arun, 192311001:Zoya]
```

Test	Predicted size	Observed	Reason
1 – same object scanned twice (faulty class)	2	2	Same reference gives the same identity hash; HashSet also checks == first. The bug is hidden.
2 – separate objects, same regNo (faulty class)	4	4	Different identity hashes; equals() is never called although it says true.
2b – same scans, corrected class	2	2	Equal objects share a hash; equals() confirms.
3 – regNo changed after insertion	1 (and lookups fail)	1; contains() false for old key, new key and the object itself	The element sits in the bucket of the old hash; lookups use the new hash. Re-adding creates a hidden duplicate (size 2).
4 – TreeSet ordered by name only	2	2	TreeSet uses compareTo, not equals: two “Arun” entries compare 0 so the second is dropped though their regNo differs.

3.4 Inquiry

- **Why duplicates appear:** equals() was overridden but hashCode() was not, which breaks contract rule 1. Test 2 proves the point: equals() returns true yet the set holds both objects.
- **How HashSet locates an element:** hash → bucket → equals() inside that bucket (Section 3.1). If the hash is inconsistent the element is searched in the wrong bucket.
- **Changing a hashed field after insertion:** the stored position becomes stale. Test 3 shows contains() returning false even for the very same object. Remedy: make key fields final (the corrected Student has private final String regNo) or remove, modify and re-add.
- **Correct class and how to verify:** override hashCode() with the same field(s) used in equals() (Objects.hash(regNo) or regNo.hashCode()). Verify with the Test 2 / 2b pair (size must drop from 4 to 2), and with a property check: for random pairs, equals ⇒equal hashCodes.

3.5 Alternative concept – TreeSet with Comparator, or HashMap keyed by regNo

DedupAlt.java shows both alternatives. A **TreeSet<Student> with Comparator.comparing(Student::regNo)** treats two scans as duplicates when the comparator returns 0, so no hashCode is needed, and the set stays sorted by regNo. A **HashMap<String,Student> keyed by regNo** uses String's own correct hashCode and equals, so the Student class does not need to implement either method; putIfAbsent keeps the first scan.

DedupAlt.java

```
import java.util.*;

public class DedupAlt {
   record S(String regNo, String name) {}
   static long sink;
   public static void main(String[] args) {
      S[] scans = { new S("192311001","Arun"), new S("192311002","Divya"), new S("192311001","Arun"), new S("192311003","Kavya"), new S("192311002","Divya") };
      Set<S> ts = new TreeSet<>(Comparator.comparing(S::regNo));
      Map<String,S> hm = new HashMap<>();
      for (S s : scans) { ts.add(s); hm.putIfAbsent(s.regNo(), s); }
      System.out.println("TreeSet(Comparator on regNo) unique = " + ts.size() + " " + ts);
      System.out.println("HashMap<String,S> keyed by regNo unique = " + hm.size() + " " + new TreeMap<>(hm).values());
      System.out.println();
      System.out.printf("%-10s %-14s %-14s %-14s%n", "n unique", "HashSet (ms)", "HashMap (ms)", "TreeSet (ms)");
      for (int n : new int[]{10_000, 100_000, 500_000}) {
         String[] ids = new String[n * 2]; Random r = new Random(5);
         for (int i = 0; i < ids.length; i++) ids[i] = "19" + (2311000 + r.nextInt(n));
         double[] t = new double[3];
         for (int rep = 0; rep < 4; rep++) {
            long a = System.nanoTime(); Set<String> h = new HashSet<>(); for (String id : ids) h.add(id); long b = System.nanoTime();
```

```
            Map<String,Integer> m = new HashMap<>(); for (String id : ids) m.putIfAbsent(id, 1); long
c = System.nanoTime();
            Set<String> tr = new TreeSet<>(); for (String id : ids) tr.add(id); long d =
System.nanoTime();
            sink += h.size() + m.size() + tr.size();
            if (rep > 0) { t[0] += (b - a) / 1e6; t[1] += (c - b) / 1e6; t[2] += (d - c) / 1e6; }
          }
          System.out.printf("%-10d %-14.2f %-14.2f %-14.2f    (2n scans each)%n", n, t[0]/3, t[1]/3,
t[2]/3);
        }
        if (sink == 3) System.out.println();
    }
}
```

Output (time for 2n scans of n unique IDs, ms)

```
TreeSet(Comparator on regNo) unique = 3 [S[regNo=192311001, name=Arun], S[regNo=192311002,
name=Divya], S[regNo=192311003, name=Kavya]]
HashMap<String,S> keyed by regNo unique = 3 [S[regNo=192311001, name=Arun],
S[regNo=192311002, name=Divya], S[regNo=192311003, name=Kavya]]

n unique   HashSet (ms)   HashMap (ms)   TreeSet (ms)
10000      4.39           5.81           13.70          (2n scans each)
100000     16.81          9.66           114.96         (2n scans each)
500000     88.56          234.57         1247.86        (2n scans each)
```

Structure	Time per scan	Space for n unique students	Notes / measured (n = 500,000)
HashSet<Student>	O(1) average (O(log n) worst case with treeified buckets)	O(n): entry + table	Fast: 88.6 ms for 1,000,000 scans; needs contract-correct equals/hashCode
HashMap<String,Student>	O(1) average	O(n): entry + key reference + table	234.6 ms in this run (likely GC and run-to-run noise; same order of magnitude as HashSet)
TreeSet with Comparator	O(log n) per scan	O(n): one tree node per element (larger per element than hash entry)	1,248 ms – about 14× slower than HashSet, but sorted order for free

The measurements follow the predicted growth: HashSet time grows roughly in proportion to the number of scans (4.4 ms for 20,000 scans, 16.8 ms for 200,000, 88.6 ms for 1,000,000), while the TreeSet is slower at every size (13.7, 115, 1,248 ms) and its per-scan cost also grows with log n. **Recommendation:** HashSet (with a correct Student class) when only uniqueness matters; TreeSet only if sorted output by regNo is also needed.

Q4 – Choosing a Map for a Pharmacy Inventory

Bloom's level: K4 – Analyse | Marks: 6

4.1 Pre-implementation analysis

Map	put	get	Iteration	Natural report	Extra work for the other report
HashMap	O(1) avg	O(1) avg	O(capacity + n), arbitrary order	neither	Code order: copy keys and sort, O(n log n). Insertion order: impossible without keeping a separate list.
LinkedHash Map	O(1) avg	O(1) avg	O(n), insertion order	insertion order	Code order: copy keys and sort, O(n log n).
TreeMap	O(log n)	O(log n)	O(n), sorted by key	code order	Insertion order: needs a separate list (O(n) extra space).

4.2 Implementation and measurements

MapBench.java stores medicine codes (MED000000 …) inserted in **shuffled** order so that insertion order is deliberately different from code order. For n = 1,000, 10,000 and 100,000 it measures (average of 5 runs, after a JIT warm-up pass): build time, 100,000 random lookups, the time to produce each report, and verifies that each report's order matches the requirement. A sorted array with Arrays.binarySearch() is measured as the alternative.

MapBench.java

```
import java.util.*;

public class MapBench {
    record Medicine(String code, int stock) {}
    static long sink;
    static final int LOOKUPS = 100_000, REPS = 5;

    static String code(int i) { return String.format("MED%06d", i); }

    public static void main(String[] args) {
        if (args.length == 0) { java.io.PrintStream o = System.out; System.setOut(new java.io.PrintStream(java.io.OutputStream.nullOutputStream())); main(new String[]{"warm-up"}); System.setOut(o); }
        // ---- functional check on a tiny data set (order requirement) ----
        String[] added = {"MED000300", "MED000100", "MED000200"};
        Map<String,Integer> h = new HashMap<>(), l = new LinkedHashMap<>(), t = new TreeMap<>();
        for (String c : added) { h.put(c, 1); l.put(c, 1); t.put(c, 1); }
        System.out.println("Order check, medicines added in order " + Arrays.toString(added));
        System.out.println("  HashMap       iteration: " + h.keySet() + "  (no guaranteed order)");
```

```
        System.out.println("  LinkedHashMap  iteration: " + l.keySet() + "  (= insertion order report)");
        System.out.println("  TreeMap        iteration: " + t.keySet() + "  (= code order report)");
        String[] arr = {"MED000100", "MED000200", "MED000300"};
        System.out.println("  Arrays.binarySearch(sorted array, MED000200) = " + Arrays.binarySearch(arr, "MED000200") + ", missing key MED000250 -> " + Arrays.binarySearch(arr, "MED000250") + " (-(insertion point)-1)");
        System.out.println();

        int[] sizes = {1_000, 10_000, 100_000};
        System.out.println("Averages of " + REPS + " runs (ms). Lookups = " + LOOKUPS + " get() calls on random existing codes.");
        System.out.printf("%-8s %-14s %12s %14s %18s %18s%n", "n", "Structure", "build", "100k lookups", "code-order report", "insert-order report");
        for (int n : sizes) {
            List<String> order = new ArrayList<>(); for (int i = 0; i < n; i++) order.add(code(i));
            Collections.shuffle(order, new Random(3));              // insertion order is NOT code order
            String[] keys = new String[LOOKUPS]; Random r = new Random(9); for (int i = 0; i < LOOKUPS; i++) keys[i] = order.get(r.nextInt(n));
            String[] types = {"HashMap", "LinkedHashMap", "TreeMap"};
            for (String ty : types) {
                double build = 0, look = 0, repCode = 0, repIns = 0; boolean okCode = true, okIns = true;
                for (int rep = 0; rep < REPS + 2; rep++) {
                    long t0 = System.nanoTime();
                    Map<String,Integer> m = ty.equals("HashMap") ? new HashMap<>() : ty.equals("LinkedHashMap") ? new LinkedHashMap<>() : new TreeMap<>();
                    List<String> sideList = ty.equals("TreeMap") || ty.equals("HashMap") ? new ArrayList<>() : null;   // extra work needed for insertion-order report
                    for (String c : order) { m.put(c, 5); if (sideList != null) sideList.add(c); }
                    long t1 = System.nanoTime();
                    for (String k : keys) sink += m.get(k);
                    long t2 = System.nanoTime();
                    // code-order report
                    StringBuilder sb = new StringBuilder();
                    if (ty.equals("TreeMap")) for (var e : m.entrySet()) sb.append(e.getKey()).append('=').append(e.getValue()).append('\n');
                    else { List<String> ks = new ArrayList<>(m.keySet()); Collections.sort(ks); for (String k : ks) sb.append(k).append('=').append(m.get(k)).append('\n'); }
                    long t3 = System.nanoTime();
                    String codeReport = sb.toString();
                    // insertion-order report
                    StringBuilder sb2 = new StringBuilder();
                    if (ty.equals("LinkedHashMap")) for (var e : m.entrySet()) sb2.append(e.getKey()).append('=').append(e.getValue()).append('\n');
                    else for (String k : sideList) sb2.append(k).append('=').append(m.get(k)).append('\n');
                    long t4 = System.nanoTime();
                    String insReport = sb2.toString();
```

```
                if (rep >= 2) { build += (t1 - t0) / 1e6; look += (t2 - t1) / 1e6; repCode += (t3 - t2) / 1e6; repIns += (t4 - t3) / 1e6; }
                // verify ordering
                List<String> sortedKeys = new ArrayList<>(order); Collections.sort(sortedKeys);
                okCode &= codeReport.startsWith(sortedKeys.get(0)) && codeReport.lines().map(s -> s.substring(0, 9)).toList().equals(sortedKeys);
                okIns &= insReport.lines().map(s -> s.substring(0, 9)).toList().equals(order);
                sink += codeReport.length() + insReport.length();
            }
            System.out.printf("%-8d %-14s %12.2f %14.2f %12.2f (%s) %12.2f (%s)%n", n, ty, build / REPS, look / REPS, repCode / REPS, okCode ? "ok" : "BAD", repIns / REPS, okIns ? "ok" : "BAD");
        }
        // Sorted array + binary search
        Medicine[] inv = new Medicine[n]; List<String> sorted = new ArrayList<>(order); Collections.sort(sorted);
        for (int i = 0; i < n; i++) inv[i] = new Medicine(sorted.get(i), 5);
        String[] codes = new String[n]; for (int i = 0; i < n; i++) codes[i] = inv[i].code();
        double look = 0, ins = 0;
        for (int rep = 0; rep < REPS + 2; rep++) {
            long a = System.nanoTime(); for (String k : keys) sink += Arrays.binarySearch(codes, k); long b = System.nanoTime();
            String[] cp = Arrays.copyOf(codes, n + 1000); int len = n; Random rr = new Random(rep);
            for (int i = 0; i < 1000; i++) {       // insert 1000 new codes keeping the array sorted
                String nk = String.format("MED%06dX", rr.nextInt(n));
                int pos = Arrays.binarySearch(cp, 0, len, nk); if (pos < 0) pos = -pos - 1;
                System.arraycopy(cp, pos, cp, pos + 1, len - pos); cp[pos] = nk; len++;
            }
            long c = System.nanoTime(); sink += len;
            if (rep >= 2) { look += (b - a) / 1e6; ins += (c - b) / 1e6; }
        }
        System.out.printf("%-8d %-14s %12s %14.2f %18s %18s     (insert of 1000 codes: %.2f ms)%n", n, "sorted array", "-", look / REPS, "free (already sorted)", "n/a", ins / REPS);
        System.out.println();
      }
      if (sink == 1) System.out.println();
   }
}
```

Output (milliseconds; “ok” = order verified)

```
Order check, medicines added in order [MED000300, MED000100, MED000200]
  HashMap        iteration: [MED000100, MED000300, MED000200]  (no guaranteed order)
  LinkedHashMap  iteration: [MED000300, MED000100, MED000200]  (= insertion order report)
  TreeMap        iteration: [MED000100, MED000200, MED000300]  (= code order report)
  Arrays.binarySearch(sorted array, MED000200) = 1, missing key MED000250 -> -3 (-(insertion point)-1)
```

```
Averages of 5 runs (ms). Lookups = 100000 get() calls on random existing codes.
n       Structure          build   100k lookups  code-order report insert-order report
1000    HashMap            0.07        5.55       1.25 (ok)       0.08 (ok)
1000    LinkedHashMap      0.06        3.35       0.34 (ok)       0.06 (ok)
1000    TreeMap            1.12       41.83       0.92 (ok)       0.29 (ok)
1000    sorted array          -       26.38 free (already sorted)          n/a   (insert of 1000 codes:
0.57 ms)

10000   HashMap            1.36        3.07       6.18 (ok)       1.93 (ok)
10000   LinkedHashMap      1.38        3.47       7.62 (ok)       0.36 (ok)
10000   TreeMap            3.87       30.01       0.56 (ok)       3.04 (ok)
10000   sorted array          -       28.43 free (already sorted)          n/a   (insert of 1000 codes:
1.22 ms)

100000  HashMap            8.73        2.57       26.90 (ok)      9.91 (ok)
100000  LinkedHashMap     13.12        3.25       58.73 (ok)      21.33 (ok)
100000  TreeMap           62.85       61.15       19.68 (ok)      68.87 (ok)
100000  sorted array          -       48.22 free (already sorted)          n/a   (insert of 1000 codes:
5.85 ms)
```

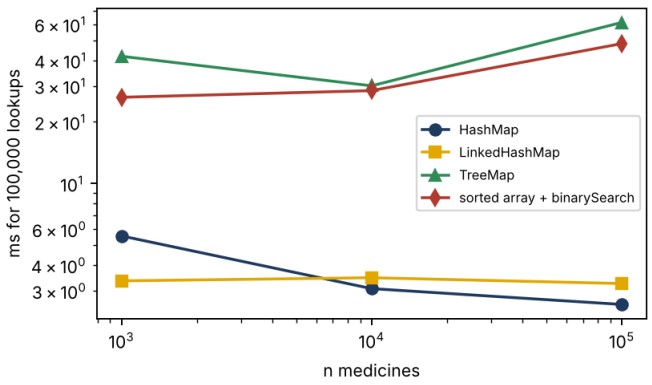


Figure 3 – 100,000 lookups at three sizes (log–log).

4.3 Scalability inquiry

Acceptability criterion: a pharmacist is working interactively, so one lookup must finish in under 1 ms (a safe fraction of the ~100 ms a person perceives as instant). Even TreeMap, the slowest map, needs 61 ms for 100,000 lookups at n = 100,000, which is about **0.6 µs per lookup**, more than 1,000× inside the criterion. HashMap and LinkedHashMap take about 0.03–0.06 µs. Every map therefore meets the response-time need; the choice is decided by the reports.

Decision: deploy **LinkedHashMap** for the stock table. It gives O(1) lookup and the insertion-order report for free. The code-order report is produced once an evening by copying and sorting the keys: 58.7 ms at 100,000 medicines, which is negligible for a nightly job. (TreeMap plus a side list for insertion order would also work, but pays O(log n) on every one of the hundreds of daily lookups and keeps two structures consistent.) The "remaining cost" is not hashing itself: it is String hashing and comparison, pointer chasing between entry objects

(cache misses) and, for TreeMap, about $\log_2(100{,}000) \approx 17$ key comparisons per lookup. Timings at n = 1,000 are noisier because the runs are shorter than the JIT and timer stabilise.

Why Hashtable is not preferred in new code: every method is synchronized, so threads serialise on one lock even for reads; it is a legacy class that predates the Collections Framework; it forbids null keys and values; and it offers no ordering. For concurrent use, ConcurrentHashMap (fine-grained locking) is the replacement; for single-threaded use, HashMap or LinkedHashMap.

4.4 Alternative concept – sorted array with Arrays.binarySearch()

Keep Medicine[] (or String[] of codes) sorted by code. Lookup is binarySearch, O(log n): the output shows 48 ms for 100,000 lookups at n = 100,000, similar to TreeMap. Insertion must find the position (O(log n)) and shift the tail with System.arraycopy, O(n): 1,000 insertions took 5.9 ms at n = 100,000. The code-order report is free because the array is already sorted (O(n) walk). Space is the most compact of all: n references, with no entry or node objects.

Structure	Lookup	Insertion	Code-order report	Space
HashMap / LinkedHashMap	O(1) avg	O(1) avg	O(n log n) sort	O(n), entry objects + table
TreeMap	O(log n)	O(log n)	O(n), already sorted	O(n), one node per entry
Sorted array + binarySearch	O(log n)	O(n) shift	O(n), already sorted	O(n), most compact

The sorted array is attractive for read-mostly data that changes rarely (the medicine list rarely gets new codes), but stock updates do not change keys, so it also works there. It loses when many new codes arrive, since each insertion is O(n). Since the pharmacy's requirements include the insertion-order report as well, the array would additionally need a second structure; LinkedHashMap remains the simplest correct choice.

Q5 – Derive a Generic Ranking Component from a Principle

Bloom's level: K4 – Analyse | Marks: 7

5.1 Deriving the design from the principle

The principle says a ranking is correct only if the comparison is **transitive** and **equal items compare as equal** (consistent). Translated into Java: the comparison must define a total preorder: for all a, b, c, sign(compare(a,b)) = −sign(compare(b,a)); compare(a,b) < 0 and compare(b,c) < 0 imply compare(a,c) < 0; and compare(a,b) = 0 implies a and b compare identically against every c. A multi-field comparison achieves this by **lexicographic composition** of per-field comparisons that are each total orders: compare the first field; only if it ties, compare the second; and so on.

Design question	Decision and reason

Design question	Decision and reason
Comparable<Team> or Comparator?	**Comparator objects.** A class can have only one natural order, but this meet needs different orders (for league table, for goal-difference award, for alphabetical lists), and the same component must rank players and colleges that we do not own or want to modify. A Comparator is passed in from outside, so Leaderboard<T> need not know anything about T.
Multi-field comparison	Comparator.comparingInt(points).reversed().thenComparing(goalDiff reversed).thenComparing(name). reversed() flips one key; thenComparing applies the next key only on ties.
Generic class	Leaderboard<T> holds a Comparator<? super T>. No bound on T is needed because the order is supplied. When an element type is naturally ordered, a bounded method <E extends Comparable<? super E>> E best(Collection<? extends E>) works.
Wildcards	Comparator<? super T> (consumer): a Comparator<Object> or Comparator<Person> can rank Teams. Collection<? extends T> (producer) in addAll(): a List<ComparableTeam> can be added to a Leaderboard<Team>. Rule: PECS – producer extends, consumer super.

5.2 Complete the Code – LeaderboardDemo

TODO 1 returns a sorted copy (new ArrayList<>(items) then sort(order)) so the stored list is never modified. TODO 2 builds the comparator chain.

LeaderboardDemo.java

```
import java.util.ArrayList;
import java.util.Comparator;
import java.util.List;

class Team {
    final String name; final int points; final int goalDiff;
    Team(String name, int points, int goalDiff) {
        this.name = name; this.points = points; this.goalDiff = goalDiff;
    }
    @Override public String toString() {
        return name + "(" + points + "," + goalDiff + ")";
    }
}

class Leaderboard<T> {
    private final List<T> items = new ArrayList<>();
    private final Comparator<? super T> order;
    Leaderboard(Comparator<? super T> order) { this.order = order; }
    void add(T item) { items.add(item); }
    List<T> ranking() {
        List<T> copy = new ArrayList<>(items);   // sorted COPY: items is not modified
```

```
        copy.sort(order);
        return copy;
    }
}

public class LeaderboardDemo {
    public static void main(String[] args) {
        Comparator<Team> byRank = Comparator.comparingInt((Team t) -> t.points).reversed()
                .thenComparing(Comparator.comparingInt((Team t) -> t.goalDiff).reversed())
                .thenComparing(t -> t.name);
        Leaderboard<Team> board = new Leaderboard<>(byRank);
        board.add(new Team("Panthers", 9, 4));
        board.add(new Team("Eagles", 9, 6));
        board.add(new Team("Cobras", 6, 1));
        board.add(new Team("Bisons", 9, 4));
        System.out.println(board.ranking());
    }
}
```

Output (matches the expected output)

```
[Eagles(9,6), Bisons(9,4), Panthers(9,4), Cobras(6,1)]
```

Why Bisons ranks above Panthers: both have 9 points and goal difference +4, so the first two keys tie. The third key, the name, decides: “Bisons” comes before “Panthers” alphabetically (B < P). Without that third key, the order of the two would depend only on their input order.

5.3 Tests: data with ties, and data built to break a faulty comparator

LeaderboardExtras.java

```
import java.util.*;

class Team {
    final String name; final int points; final int goalDiff;
    Team(String n, int p, int g) { name = n; points = p; goalDiff = g; }
    @Override public String toString() { return name + "(" + points + "," + goalDiff + ")"; }
}
class Board<T> {                                    // generic, reusable, wildcard-friendly
    private final List<T> items = new ArrayList<>();
    private final Comparator<? super T> order;
    Board(Comparator<? super T> order) { this.order = order; }
    void add(T t) { items.add(t); }
    void addAll(Collection<? extends T> more) { items.addAll(more); }          // ? extends T : producer
    List<T> ranking() { List<T> c = new ArrayList<>(items); c.sort(order); return c; }
    static <E extends Comparable<? super E>> E best(Collection<? extends E> c) { return
Collections.max(c); }   // bounded type
}
```

```
class Player { final String name; final double rating; Player(String n, double r) { name = n; rating = r; } public String toString() { return name + "(" + rating + ")"; } }
class ComparableTeam extends Team implements Comparable<Team> {
    ComparableTeam(String n, int p, int g) { super(n, p, g); }
    @Override public int compareTo(Team o) {
        int c = Integer.compare(o.points, points); if (c != 0) return c;
        c = Integer.compare(o.goalDiff, goalDiff); if (c != 0) return c;
        return name.compareTo(o.name);
    }
}

public class LeaderboardExtras {
    static final Comparator<Team> GOOD = Comparator.comparingInt((Team t) -> t.points).reversed()
            .thenComparing(Comparator.comparingInt((Team t) -> t.goalDiff).reversed()).thenComparing(t -> t.name);
    static final Comparator<Team> NO_TIEBREAK = Comparator.comparingInt((Team t) -> t.points).reversed()
            .thenComparing(Comparator.comparingInt((Team t) -> t.goalDiff).reversed());
    static final Comparator<Team> SUBTRACT = (a, b) -> {          // faulty: comparison by subtraction
        int c = b.points - a.points; if (c != 0) return c;
        c = b.goalDiff - a.goalDiff; if (c != 0) return c;
        return a.name.compareTo(b.name);
    };
    static List<Team> sorted(List<Team> in, Comparator<Team> c) { List<Team> l = new ArrayList<>(in); l.sort(c); return l; }

    public static void main(String[] args) {
        System.out.println("== Data set A: several ties at every level ==");
        List<Team> a = List.of(new Team("Tigers", 12, 5), new Team("Lions", 12, 5), new Team("Wolves", 12, 5),
            new Team("Hawks", 12, 2), new Team("Bears", 9, 7), new Team("Foxes", 9, 7), new Team("Rams", 9, -3), new Team("Owls", 3, 0));
        List<Team> shuffled = new ArrayList<>(a); Collections.shuffle(shuffled, new Random(11));
        System.out.println("Input (shuffled): " + shuffled);
        System.out.println("Ranking          : " + sorted(shuffled, GOOD));
        System.out.println("Same result for 5 different shuffles: " + sameForShuffles(a));
        System.out.println();
        System.out.println("== Data set B: designed to break faulty comparators ==");
        List<Team> b = List.of(new Team("Alpha", 5, 1_500_000_000), new Team("Beta", 5, -1_500_000_000), new Team("Gamma", 5, 0));
        System.out.println("Correct comparator : " + sorted(b, GOOD));
        System.out.println("Subtraction version: " + sorted(b, SUBTRACT) + "   <- goalDiff 1.5e9 - (-1.5e9) overflows int");
        System.out.println("  compare(Alpha,Beta) by subtraction = " + SUBTRACT.compare(b.get(0), b.get(1)) + " (should be negative: Alpha ranks first); compare(Beta,Alpha) = " + SUBTRACT.compare(b.get(1), b.get(0)));
```

```
        List<Team> c = List.of(new Team("Zulu", 7, 3), new Team("Echo", 7, 3), new Team("Kilo", 7, 3));
        System.out.println("No-name-tiebreak    : " + sorted(c, NO_TIEBREAK) + "   <- ties keep input order (stable) not alphabetical");
        System.out.println("Correct (with name): " + sorted(c, GOOD));
        List<Team> cr = new ArrayList<>(c); Collections.reverse(cr);
        System.out.println("No-tiebreak on reversed input: " + sorted(cr, NO_TIEBREAK) + "   <- result depends on input order, so the ranking is not well defined");
        System.out.println();
        System.out.println("== Alternatives give the same ranking ==");
        List<ComparableTeam> ct = new ArrayList<>(); for (Team t : a) ct.add(new ComparableTeam(t.name, t.points, t.goalDiff));
        Collections.shuffle(ct, new Random(2)); Collections.sort(ct);
        System.out.println("Comparable<Team> + Collections.sort: " + ct);
        List<Team> three = new ArrayList<>(shuffled);
        three.sort(Comparator.comparing((Team t) -> t.name));
        three.sort(Comparator.comparingInt((Team t) -> t.goalDiff).reversed());
        three.sort(Comparator.comparingInt((Team t) -> t.points).reversed());
        System.out.println("Three stable sorts                : " + three);
        System.out.println("All three identical: " + (sorted(shuffled, GOOD).toString().equals(ct.toString()) && ct.toString().equals(three.toString())));
        System.out.println();
        System.out.println("== Reuse with other types / wildcards ==");
        Board<Player> pb = new Board<>(Comparator.comparingDouble((Player p) -> p.rating).reversed().thenComparing(p -> p.name));
        pb.add(new Player("Ravi", 8.5)); pb.add(new Player("Asha", 9.1)); pb.add(new Player("Kiran", 8.5));
        System.out.println("Players: " + pb.ranking());
        Board<Team> tb = new Board<>(GOOD); tb.addAll(List.of(new ComparableTeam("X", 3, 0), new ComparableTeam("Y", 6, 1)));   // List<ComparableTeam> into Board<Team>
        System.out.println("addAll(List<? extends Team>) with ComparableTeam objects: " + tb.ranking());
        System.out.println("Board.best(List<Integer>) = " + Board.best(List.of(4, 9, 2)));
        Comparator<Object> byToString = Comparator.comparing(Object::toString);
        Board<Team> viaSuper = new Board<>(byToString);    // Comparator<? super T> accepts a Comparator<Object>
        viaSuper.add(new Team("Q", 1, 1)); System.out.println("Comparator<Object> accepted by Board<Team> (? super T): " + viaSuper.ranking());
        System.out.println();
        System.out.println("== Timing: Comparator chain (1 sort) vs three stable sorts, n teams, ms avg of 5 ==");
        for (int n : new int[]{10_000, 100_000, 500_000}) {
            Random r = new Random(1); List<Team> big = new ArrayList<>();
            for (int i = 0; i < n; i++) big.add(new Team("T" + r.nextInt(n), r.nextInt(30), r.nextInt(40) - 20));
            double t1 = 0, t3 = 0;
            for (int rep = 0; rep < 7; rep++) {
```

```
            List<Team> x = new ArrayList<>(big); long s = System.nanoTime(); x.sort(GOOD); long m = System.nanoTime();
            List<Team> y = new ArrayList<>(big); y.sort(Comparator.comparing((Team t) -> t.name)); y.sort(Comparator.comparingInt((Team t) -> t.goalDiff).reversed()); y.sort(Comparator.comparingInt((Team t) -> t.points).reversed()); long e = System.nanoTime();
            if (rep >= 2) { t1 += (m - s) / 1e6; t3 += (e - m) / 1e6; }
          }
          System.out.printf("n=%-8d chain: %8.2f ms   three sorts: %8.2f ms%n", n, t1 / 5, t3 / 5);
        }
    }
    static boolean sameForShuffles(List<Team> a) {
        String ref = sorted(a, GOOD).toString();
        for (int s = 0; s < 5; s++) { List<Team> x = new ArrayList<>(a); Collections.shuffle(x, new Random(100 + s)); if (!sorted(x, GOOD).toString().equals(ref)) return false; }
        return true;
    }
}
```

Output

```
== Data set A: several ties at every level ==
Input (shuffled): [Lions(12,5), Rams(9,-3), Bears(9,7), Wolves(12,5), Tigers(12,5), Hawks(12,2), Owls(3,0), Foxes(9,7)]
Ranking           : [Lions(12,5), Tigers(12,5), Wolves(12,5), Hawks(12,2), Bears(9,7), Foxes(9,7), Rams(9,-3), Owls(3,0)]
Same result for 5 different shuffles: true

== Data set B: designed to break faulty comparators ==
Correct comparator : [Alpha(5,1500000000), Gamma(5,0), Beta(5,-1500000000)]
Subtraction version: [Gamma(5,0), Beta(5,-1500000000), Alpha(5,1500000000)]   <- goalDiff 1.5e9 - (-1.5e9) overflows int
 compare(Alpha,Beta) by subtraction = 1294967296 (should be negative: Alpha ranks first); compare(Beta,Alpha) = -1294967296
No-name-tiebreak    : [Zulu(7,3), Echo(7,3), Kilo(7,3)]    <- ties keep input order (stable) not alphabetical
Correct (with name): [Echo(7,3), Kilo(7,3), Zulu(7,3)]
No-tiebreak on reversed input: [Kilo(7,3), Echo(7,3), Zulu(7,3)]   <- result depends on input order, so the ranking is not well defined

== Alternatives give the same ranking ==
Comparable<Team> + Collections.sort: [Lions(12,5), Tigers(12,5), Wolves(12,5), Hawks(12,2), Bears(9,7), Foxes(9,7), Rams(9,-3), Owls(3,0)]
Three stable sorts                 : [Lions(12,5), Tigers(12,5), Wolves(12,5), Hawks(12,2), Bears(9,7), Foxes(9,7), Rams(9,-3), Owls(3,0)]
All three identical: true

== Reuse with other types / wildcards ==
```

```
Players: [Asha(9.1), Kiran(8.5), Ravi(8.5)]
addAll(List<? extends Team>) with ComparableTeam objects: [Y(6,1), X(3,0)]
Board.best(List<Integer>) = 9
Comparator<Object> accepted by Board<Team> (? super T): [Q(1,1)]

== Timing: Comparator chain (1 sort) vs three stable sorts, n teams, ms avg of 5 ==
n=10000    chain:     6.96 ms   three sorts:    17.44 ms
n=100000   chain:    49.82 ms   three sorts:    93.90 ms
n=500000   chain:   458.67 ms   three sorts:   642.45 ms
```

5.4 Correctness inquiry – reasoning, not just output

Let key(t) = (−points, −goalDiff, name). The comparator orders teams by comparing key(t) lexicographically. Each component is compared with a total order on its own type (integers, strings), and **lexicographic composition of total orders is a total order**. Proof of transitivity: suppose A < B and B < C. Let i be the first component where A and B differ (A smaller there) and j the first where B and C differ. If i < j, then A and C agree before i and A is smaller at i, so A < C. If j < i, A and C agree before j (A equals B there) and C is larger at j, so A < C. If i = j the component order is transitive, so A < C. The same argument covers equality (all components equal). Hence ranking is consistent. The tests agree: five random shuffles give the identical ranking, and the Comparable and three-sort versions give the same output.

Why the faulty versions fail, by reasoning: (1) Subtraction b.goalDiff − a.goalDiff is not a valid comparison, because the subtraction overflows int when the operands are far apart: 1,500,000,000 − (−1,500,000,000) = 3,000,000,000, which wraps to −1,294,967,296. The result has the wrong sign: compare(Alpha, Beta) came out positive (+1,294,967,296) although Alpha must rank first, and the sorted list placed Alpha last, after Gamma and Beta. (2) A comparator without the name tiebreak leaves tied teams “equal”: they keep input order (stable sort), so the same data in reversed order yields a different ranking, i.e. the output is not a function of the data. The program output shows both failures.

5.5 Alternative concept – Comparable<Team> or three stable sorts

With Team implements Comparable<Team> the same lexicographic logic lives inside compareTo; Collections.sort(list) and TreeSet work without extra arguments, but Team is now permanently tied to one ordering. With **three stable sorts** the list is sorted by name, then by goal difference, then by points; because the sorts are stable, earlier orderings survive within ties, so the final result is the same (output confirms identical lists). It is correct only when the sorts are applied in exactly this reverse order of significance and the sort algorithm is stable, which is easy to get wrong.

Approach	Number of O(n log n) sorts	Extra space	Reuse for players / colleges
Comparator chain (used)	1	O(n) for the sorted copy (merge-sort temp array)	Best: each type just supplies its own Comparator; Leaderboard<T> is unchanged

Approach	Number of O(n log n) sorts	Extra space	Reuse for players / colleges
Comparable<Team>	1	O(n) as above	Poor: every class needs its own compareTo and only one order each
Three stable sorts	3	O(n) temp per sort (not retained)	Poor: order of sorts is an easy-to-break convention

Measured on n teams (milliseconds): n = 10,000 → 7.0 vs 17.4; n = 100,000 → 49.8 vs 93.9; n = 500,000 → 458.7 vs 642.5. The single comparator sort is faster than three sorts at every size, matching the 1-sort versus 3-sorts prediction (the ratio is below 3×, probably because the second and third sorts start from partially ordered input, which the merge-based sort handles faster).

Q6 – Priority, Order and Fairness in a Queue

Bloom's level: K5 – Evaluate | Marks: 8

6.1 Design reasoning (before code)

- **Why a plain Queue is not enough:** LinkedList/ArrayDeque serve strictly in arrival order (FIFO). A critical patient who arrives last would wait behind everyone, so the rule "most severe first" cannot be expressed by arrival order alone.
- **PriorityQueue with a Comparator:** a binary min-heap ordered by the comparator. Comparing severity first puts the lowest severity number (most critical) at the head. add() is O(log n) (sift-up); poll() is O(log n) (sift-down); peek() is O(1).
- **Preserving arrival order for equal severities:** a heap is not stable – elements that compare equal may come out in any order. The fix is to make "equal" impossible: give every patient a unique, increasing arrival counter and use it as the second key. Then (severity, arrival) is a strict total order and the earliest arrival wins every tie.
- **Removing and displaying:** poll() removes the next patient. A PriorityQueue's toString() and iterator() expose the internal heap array, which is only partially ordered (each parent ≤ its children), so to display the waiting list in treatment order, copy the queue and poll the copy, or sort a copy.

6.2 Complete the Code – Triage

Triage.java

```
import java.util.Comparator;
import java.util.PriorityQueue;

class Patient {
   final String name; final int severity; final long arrival;
   Patient(String name, int severity, long arrival) {
      this.name = name; this.severity = severity; this.arrival = arrival;
   }
```

```
    @Override public String toString() { return name + "(S" + severity + ")"; }
}

public class Triage {
    private long counter = 0;
    private final PriorityQueue<Patient> queue = new PriorityQueue<>(
            Comparator.comparingInt((Patient p) -> p.severity)      // lowest number = most severe first
                    .thenComparingLong(p -> p.arrival));          // equal severity: earliest arrival first

    void admit(String name, int severity) {
        queue.add(new Patient(name, severity, counter++));
    }
    Patient next() { return queue.poll(); }

    public static void main(String[] args) {
        Triage er = new Triage();
        er.admit("Ravi", 3); er.admit("Meena", 1); er.admit("John", 3);
        er.admit("Asha", 5); er.admit("Kumar", 1); er.admit("Lata", 2);
        StringBuilder order = new StringBuilder();
        Patient p;
        while ((p = er.next()) != null) order.append(p).append(' ');
        System.out.println(order.toString().trim());
    }
}
```

Output (matches the expected output)

```
Meena(S1) Kumar(S1) Lata(S2) Ravi(S3) John(S3) Asha(S5)
```

How the arrival counter keeps equal severities in order: counter++ gives Ravi 0, Meena 1, John 2, Asha 3, Kumar 4, Lata 5. For the two severity-3 patients the first key ties, so the comparator falls to the arrival key: Ravi (0) is less than John (2), so Ravi is served first. The same applies to Meena (1) and Kumar (4) at severity 1. Counter values never repeat, so no two patients ever compare equal.

6.3 Hand-verified correctness check (8 patients) and the PriorityQueue printing problem

The 8 patients arrive in this order: Asha S5, Dev S4, Ravi S3, John S3, Priya S2, Lata S2, Meena S1, Kumar S1. Worked by hand: first all severity 1 by arrival (Meena, Kumar), then severity 2 (Priya, Lata), severity 3 (Ravi, John), severity 4 (Dev), severity 5 (Asha). The program output below matches that order. The same output shows what happens when the queue is printed directly: the heap array is [Meena, Kumar, Priya, Ravi, John, Dev, Lata, Asha], with Lata (S2) after Dev (S4) and John (S3), which is **not** the treatment order. A heap guarantees only that every parent is not larger than its children, not that the array is sorted.

TriageTest.java

```
import java.util.*;

public class TriageTest {
```

```
    record P(String name, int sev, long arr) { public String toString() { return name + "(S" + sev + ")"; }
}
    static final Comparator<P> CMP = Comparator.comparingInt(P::sev).thenComparingLong(P::arr);
    static long sink;

    public static void main(String[] args) {
        System.out.println("== Hand-verified instance (8 patients, arrival order 1..8) ==");
        String[][] in =
{{"Asha","5"},{"Dev","4"},{"Ravi","3"},{"John","3"},{"Priya","2"},{"Lata","2"},{"Meena","1"},{"
Kumar","1"}};
        PriorityQueue<P> pq = new PriorityQueue<>(CMP); long c = 0;
        for (String[] r : in) pq.add(new P(r[0], Integer.parseInt(r[1]), c++));
        System.out.println("Hand-worked treatment order : Meena(S1) Kumar(S1) Priya(S2) Lata(S2)
Ravi(S3) John(S3) Dev(S4) Asha(S5)");
        System.out.println("System.out.println(pq)       : " + pq + "   <- internal heap array order, NOT
treatment order");
        StringBuilder sb = new StringBuilder(); P p; while ((p = pq.poll()) != null) sb.append(p).append('
');
        String actual = sb.toString().trim();
        System.out.println("poll() until empty          : " + actual);
        System.out.println("Matches hand calculation       : " + actual.equals("Meena(S1) Kumar(S1)
Priya(S2) Lata(S2) Ravi(S3) John(S3) Dev(S4) Asha(S5)"));
        System.out.println();

        System.out.println("== Cost: admit n patients (random severity), then serve all n; ms, average of
3 runs ==");
        System.out.printf("%-9s %-26s %-14s %-14s%n", "n", "Design", "admit all", "serve all");
        for (int n : new int[]{1_000, 10_000, 40_000}) {
            int[] sev = new int[n]; Random r = new Random(4); for (int i = 0; i < n; i++) sev[i] = 1 +
r.nextInt(5);
            for (String d : new String[]{"PriorityQueue (heap)", "Sorted LinkedList", "5 FIFO queues
(array)"}) {
                double ta = 0, ts = 0;
                for (int rep = 0; rep < 4; rep++) {
                    long a0, a1, s1;
                    if (d.startsWith("Priority")) {
                        PriorityQueue<P> q = new PriorityQueue<>(CMP); a0 = System.nanoTime();
                        for (int i = 0; i < n; i++) q.add(new P("p", sev[i], i)); a1 = System.nanoTime();
                        while (q.poll() != null) sink++; s1 = System.nanoTime();
                    } else if (d.startsWith("Sorted")) {
                        LinkedList<P> q = new LinkedList<>(); a0 = System.nanoTime();
                        for (int i = 0; i < n; i++) {
                            P x = new P("p", sev[i], i); ListIterator<P> it = q.listIterator();
                            while (it.hasNext()) { if (CMP.compare(it.next(), x) > 0) { it.previous(); break; } }
                            it.add(x);
                        }
                        a1 = System.nanoTime();
```

```
                    while (q.pollFirst() != null) sink++; s1 = System.nanoTime();
                } else {
                    @SuppressWarnings("unchecked") ArrayDeque<P>[] q = new ArrayDeque[5]; for (int i = 0; i < 5; i++) q[i] = new ArrayDeque<>();
                    a0 = System.nanoTime(); for (int i = 0; i < n; i++) q[sev[i] - 1].addLast(new P("p", sev[i], i)); a1 = System.nanoTime();
                    for (int lvl = 0; lvl < 5; ) { if (q[lvl].pollFirst() == null) lvl++; else sink++; } s1 = System.nanoTime();
                }
                if (rep > 0) { ta += (a1 - a0) / 1e6; ts += (s1 - a1) / 1e6; }
            }
            System.out.printf("%-9d %-26s %-14.2f %-14.2f%n", n, d, ta / 3, ts / 3);
        }
    }
    System.out.println();
    System.out.println("== Heap only, larger sizes (admit all + serve all, ms, avg of 3) ==");
    for (int n : new int[]{5_000, 100_000, 1_000_000}) {
        Random r = new Random(8); double t = 0;
        for (int rep = 0; rep < 4; rep++) {
            PriorityQueue<P> q = new PriorityQueue<>(CMP); long a = System.nanoTime();
            for (int i = 0; i < n; i++) q.add(new P("p", 1 + r.nextInt(5), i)); while (q.poll() != null) sink++;
            if (rep > 0) t += (System.nanoTime() - a) / 1e6;
        }
        System.out.printf("n=%-9d total %8.2f ms  -> %.3f microseconds per patient (admit + serve)%n", n, t / 3, t / 3 * 1000 / n);
    }
    if (sink == 5) System.out.println();
  }
}
```

Output

```
== Hand-verified instance (8 patients, arrival order 1..8) ==
Hand-worked treatment order : Meena(S1) Kumar(S1) Priya(S2) Lata(S2) Ravi(S3) John(S3) Dev(S4) Asha(S5)
System.out.println(pq)      : [Meena(S1), Kumar(S1), Priya(S2), Ravi(S3), John(S3), Dev(S4), Lata(S2), Asha(S5)]  <- internal heap array order, NOT treatment order
poll() until empty          : Meena(S1) Kumar(S1) Priya(S2) Lata(S2) Ravi(S3) John(S3) Dev(S4) Asha(S5)
Matches hand calculation    : true

== Cost: admit n patients (random severity), then serve all n; ms, average of 3 runs ==
n         Design                    admit all      serve all
1000      PriorityQueue (heap)       0.38           2.16
1000      Sorted LinkedList         12.06           0.01
1000      5 FIFO queues (array)      0.11           0.08
```

```
10000    PriorityQueue (heap)     1.97        2.97
10000    Sorted LinkedList        192.76      0.06
10000    5 FIFO queues (array)    1.93        1.82
40000    PriorityQueue (heap)     6.37        15.82
40000    Sorted LinkedList        6156.31     1.13
40000    5 FIFO queues (array)    1.29        0.06

== Heap only, larger sizes (admit all + serve all, ms, avg of 3) ==
n=5000      total    1.36 ms  -> 0.271 microseconds per patient (admit + serve)
n=100000    total   31.45 ms  -> 0.314 microseconds per patient (admit + serve)
n=1000000   total  424.10 ms  -> 0.424 microseconds per patient (admit + serve)
```

6.4 Growth investigation – prediction versus measurement

Prediction: admitting a patient costs O(log n) (sift-up), serving the next costs O(log n) (sift-down), so n admissions plus n services cost O(n log n). For the alternatives: a sorted LinkedList costs O(n) per admission (linear scan to the position) and O(1) per service, so $O(n^2)$ for n patients; an array of five FIFO queues costs O(1) for both (the number of levels is a constant 5).

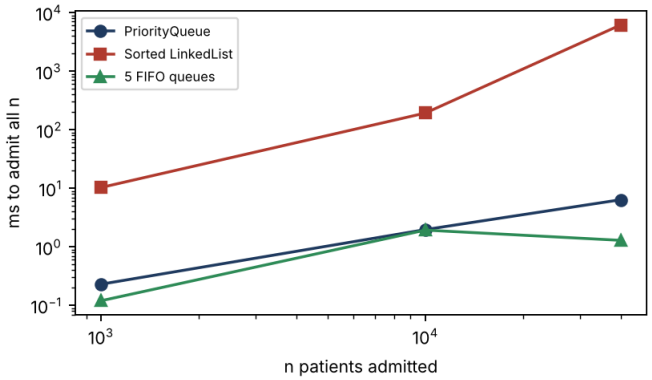


Figure 4 – Time to admit n patients for the three designs (log–log).

n	PriorityQueue admit / serve (ms)	Sorted LinkedList admit / serve (ms)	5 FIFO queues admit / serve (ms)
1,000	0.23 / 2.82	10.31 / 0.02	0.12 / 0.08
10,000	1.97 / 2.97	192.76 / 0.06	1.93 / 1.82
40,000	6.37 / 15.82	6,156.31 / 1.13	1.29 / 0.06

Validation: the sorted LinkedList grows by about 19× when n grows 10× (10 → 193 ms) and 32× when n grows 4× (193 → 6,156 ms): faster than linear, consistent with $O(n^2)$ for n insertions. The heap grows roughly n log n (6.4 ms at 40,000). The FIFO design is flat and fastest to admit. Heap timings in the first row are dominated by JIT warm-up and the small cells at n = 1,000 and 10,000 contain noise (for example the 1.82 ms serve time of the FIFO design at n = 10,000 is a garbage-collection pause). The extended heap test shows: 5,000 patients 1.36 ms; 100,000 patients 31.5 ms; 1,000,000 patients 424 ms, which is 0.27–0.42 µs per patient (admit + serve).

6.5 Starvation and a fair rule

With strict priority, a severity-5 patient is served only when no patient of severity 1–4 is waiting. If severe cases keep arriving faster than they are cleared, that patient waits indefinitely: **starvation**. The experiment in Q7 reproduces it: when one critical patient arrives every tick, the waiting severity-5 patients are never treated in 3,000 ticks. A fair rule is **aging**: the longer a patient waits, the more urgent they become. For example, every W minutes of waiting is worth one severity level, so a severity-5 patient waits at most 4W minutes longer than a later severity-1 arrival. Q7 derives an efficient implementation.

6.6 Deployment decision – 5,000 patients per day

5,000 patients per day is about 0.06 patients per second. The heap handles 1,000,000 admit-and-serve pairs in 424 ms, so a whole day's traffic would cost roughly 2 ms: the data structure is **not** the bottleneck, by about five orders of magnitude. The real limits are doctors' capacity and fairness. Therefore **the PriorityQueue design can serve a city hospital of this size, provided the aging rule is added** (strict priority is unsafe for minor cases). The sorted LinkedList would also be fast enough at this daily volume (about 6 seconds for 40,000 admissions suggests a few ms for the realistic queue length of tens of patients), but it degrades quadratically in a mass-casualty surge when the queue holds thousands, which is exactly when speed matters most.

6.7 Alternative concept – sorted LinkedList or five FIFO queues

Sorted LinkedList: admit by scanning with a ListIterator to the first element that is greater, then it.add(x); serve with pollFirst(). **Array of five FIFO queues:** ArrayDeque<Patient>[5], admit to q[severity−1].addLast, serve by scanning levels 1 to 5 for the first non-empty queue and pollFirst(). FIFO inside each queue gives arrival order with no counter needed.

Design	Admit	Serve next	Space	Comment
PriorityQueue (heap)	O(log n)	O(log n)	O(n)	Works for any number of priority values or a custom comparator
Sorted LinkedList	O(n)	O(1)	O(n) – node per patient	Quadratic total admission cost (measured)
5 FIFO queues	O(1)	O(1) (at most 5 checks)	O(n) + 5 queue headers	Fastest, but only for a small fixed set of levels; aging needs patient moves between queues

Q7 – Diagnose, Redesign and Validate (Day-4 Design Challenge)

Bloom's level: K5 – Evaluate | Marks: 13

Selected solution: the Q6 triage queue, when the requirement changes from "serve by severity" to "serve by severity but never starve a minor patient". This is a requirement change that exposes a limitation in the original design, and the obvious fix also turns out to be costly.

A. Diagnose the bottleneck (2 marks)

The original solution (strict priority PriorityQueue) fails on the new requirement: in the experiment, n severity-5 patients are waiting and one critical patient arrives every tick. The strict design never treats a single waiting patient in 3,000 ticks ("none (starved)"). The natural repair, **fix 1**, is to periodically re-scan every waiting patient, recompute its effective severity from its waiting time, and rebuild the heap. The measurements diagnose where the time goes:

n waiting	Fix 1: patients scanned	Fix 1: time (ms)	Strict priority: time (ms)
1,000	300,300	25.7	1.9
10,000	3,000,300	83.2	0.4
100,000	30,000,300	468.1	0.3

- **Where the time is spent:** in the periodic scan-and-rebuild, not in the queue operations themselves (the other designs finish the same 3,000 ticks in about 1 ms).
- **Which operation grows:** the rescan, once every 10 ticks, costs O(n) and touches every waiting patient: 300 scans × n patients (30,000,300 visits at n = 100,000), i.e. O(T/10 · n) in total.
- **Unnecessary repeated work:** yes. Almost all the scan work re-confirms an order that has not changed: re-computing keys and re-creating the heap for patients whose relative order is the same as ten ticks ago.
- **Class characteristic that causes it:** PriorityQueue orders elements by their comparator key **at the moment they are inserted or sifted**. It has no decrease-key operation, and mutating a key of an element already in the heap silently corrupts the heap invariant. So changing priorities over time forces removal and re-insertion, or a full rebuild: O(n) per change batch.

B. Design objective (2 marks)

The redesign must: (1) eliminate starvation – every severity-5 patient is treated within a bounded time (a stated policy: at most (5−1)·W extra ticks after later critical arrivals, W = 200); (2) keep equal-severity patients in arrival order; (3) keep the comparator consistent (transitive, equal means equal, per Q5); (4) make **patients scanned = 0** and keep admit and serve at O(log n); (5) produce the same treatment behaviour as fix 1 in the experiment (same first-treated tick and same counts).

C. Revised solution (4 marks)

Key insight: if every patient's urgency improves at the same rate (one severity level per W ticks of waiting), then the relative order of two waiting patients never changes as time passes – both improve equally. A quantity that never changes after admission can be computed **once**, at admission, and used as the heap key. Define for each patient:

Aging key

```
key(patient) = arrivalTime + severity * W        // computed once, at admission
order        = by key ascending, then by arrivalTime ascending   (strict total order)
```

A severity-1 patient arriving at time t has key t + W; a severity-5 patient who arrived at time 0 has key 5W. The older minor patient is served before any newcomer whose key is larger, i.e. once t + W > 5W, t > 4W. Each level of severity is worth W ticks of waiting. Among patients of the same severity the key order equals arrival order, so FIFO is preserved without any extra state.

Question	Answer
What changed	Only the comparator and one stored number (the key). The structure is still a PriorityQueue; no scanning thread or rebuild step exists.
Property exploited	Uniform aging preserves relative order, so a static key is equivalent to continuously updated priorities (Section C above).
Operations avoided	The O(n) rescan every 10 ticks and the heap rebuild; the per-tick re-computation of effective severity.
Extra memory / preprocessing	One long key per patient (computed in O(1) at admission; can also be a field computed on demand in the comparator). No other preprocessing.
New trade-offs	W becomes a policy parameter that must be chosen carefully; aging cannot be turned off for a single patient; a very large backlog of aged minor patients can delay new critical arrivals (see F). Time units must be monotonic.

Pseudocode of the redesign

```
ALGORITHM AdmitPatient(name, severity, now)
BEGIN
   key <- now + severity * W
   queue.add(Patient(name, severity, now, key))       // O(log n)
END

ALGORITHM Compare(a, b)
BEGIN
   IF a.key <> b.key THEN RETURN compare(a.key, b.key)
   RETURN compare(a.arrival, b.arrival)              // strict total order
END

ALGORITHM ServeNext()
BEGIN
   RETURN queue.poll()                          // O(log n); no scan, no rebuild
END
```

Design representation

```
          +--------------------+       +---------------------------+
 admit ---> |  Patient           |       |  PriorityQueue<Patient>   |
          |  severity, arrival |  --->  |  comparator: key, arrival  |  ---> poll()
          |  key = arrival + sev*W      +---------------------------+
          +--------------------+       (no rescan thread, no rebuild)
```

D. Prediction before implementing (1 mark)

(1) Fix 1 and the redesign will treat the first waiting severity-5 patient at the same tick, 800 (= 4·W) in this scenario, and will treat the same number of minor patients. (2) The redesign will scan **0** patients and run in time independent of n except for the O(log n) heap operations; fix 1 will scan about T/10 × n

patients. (3) I expect the improvement to **grow with n**: roughly proportional to n/log n, so at n = 100,000 more than a 100× speed-up, at n = 1,000 only a modest one.

E. Implementation and validation (2 marks)

AgingRedesign.java runs the three designs (strict priority, fix 1 with re-scan every 10 ticks, and the redesign) under identical conditions for n = 1,000, 10,000 and 100,000 waiting minor patients; each figure is an average of 3 runs after a JIT warm-up. It then checks the redesign's comparator: equal-severity FIFO, transitivity on 200,000 random triples.

AgingRedesign.java

```
import java.util.*;

public class AgingRedesign {
   static final int AGE_STEP = 200;       // naive rule: every 200 ticks of waiting, severity improves by one level
   static final int W = 200;              // redesign: one severity level is worth W ticks of waiting
   static final int RESCAN_EVERY = 10, TICKS = 3000;
   static long scanned, sink;

   static final class P {
      final int severity; final long arrival; int eff; boolean preloaded;
      P(int s, long a, boolean pre) { severity = s; arrival = a; eff = s; preloaded = pre; }
   }
   static class Result { String name; double ms; long scanned; long firstOldServed = -1; long oldServed; long maxWaitCritical; long maxWaitOld; long critWaiting; }

   // ---- Original: strict priority, no aging ----
   // ---- Original+naive fix: every RESCAN_EVERY ticks, re-scan ALL waiting patients, recompute severity and rebuild the heap ----
   // ---- Redesign: fixed key = arrival + severity*W computed once at admission; heap never re-scanned ----
   static Result run(String mode, int n) {
      Comparator<P> cmp = switch (mode) {
         case "redesign" -> Comparator.comparingLong((P p) -> p.arrival + (long) p.severity * W).thenComparingLong(p -> p.arrival);
         default -> Comparator.comparingInt((P p) -> p.eff).thenComparingLong(p -> p.arrival);
      };
      PriorityQueue<P> q = new PriorityQueue<>(cmp);
      for (int i = 0; i < n; i++) q.add(new P(5, 0, true));          // n minor patients already waiting at tick 0
      Result r = new Result(); r.name = mode; scanned = 0;
      long t0 = System.nanoTime();
      for (int t = 1; t <= TICKS; t++) {
         q.add(new P(1, t, false));                                  // one critical patient arrives every tick
         if (mode.equals("naive") && t % RESCAN_EVERY == 0) {
            List<P> all = new ArrayList<>(q);
```

```
            for (P p : all) { p.eff = Math.max(1, p.severity - (int) ((t - p.arrival) / AGE_STEP)); scanned++; }
            q = new PriorityQueue<>(cmp); q.addAll(all);
          }
          P s = q.poll();                                  // doctor serves one patient per tick
          long wait = t - s.arrival;
          if (s.preloaded) { r.oldServed++; if (r.firstOldServed < 0) r.firstOldServed = t; r.maxWaitOld = Math.max(r.maxWaitOld, wait); }
          else r.maxWaitCritical = Math.max(r.maxWaitCritical, wait);
        }
        r.ms = (System.nanoTime() - t0) / 1e6; for (P p : q) if (!p.preloaded) r.critWaiting++; r.scanned = scanned; sink += q.size();
        return r;
    }

    public static void main(String[] args) {
        run("redesign", 1000); run("naive", 1000); run("strict", 1000);     // JIT warm-up
        System.out.println("Scenario: n minor (severity 5) patients waiting at tick 0; then every tick one critical (severity 1) patient arrives and the doctor treats one patient.");
        System.out.println("Run for " + TICKS + " ticks. 'first old served' = tick at which the first of the waiting severity-5 patients is treated (none = starved).");
        System.out.printf("%-9s %-28s %9s %15s %16s %11s %13s %14s%n", "n waiting", "Design", "time (ms)", "patients scanned", "first old served", "old treated", "crit. treated", "crit. waiting at end");
        for (int n : new int[]{1_000, 10_000, 100_000}) {
            for (String m : new String[]{"strict", "naive", "redesign"}) {
                double ms = 0; Result last = null;
                for (int rep = 0; rep < 3; rep++) { last = run(m, n); ms += last.ms; }
                String label = m.equals("strict") ? "strict priority (orig.)" : m.equals("naive") ? "re-scan + rebuild (fix 1)" : "static aging key (redesign)";
                System.out.printf("%-9d %-28s %9.1f %15d %16s %11d %13d %14d%n", n, label, ms / 3, last.scanned, last.firstOldServed < 0 ? "none (starved)" : String.valueOf(last.firstOldServed), last.oldServed, TICKS - last.oldServed, last.critWaiting);
            }
        }
        // property checks on redesign
        System.out.println();
        System.out.println("Validation of the redesign (comparator properties and fairness):");
        Comparator<P> c = Comparator.comparingLong((P p) -> p.arrival + (long) p.severity * W).thenComparingLong(p -> p.arrival);
        Random rnd = new Random(1); boolean fifo = true, trans = true;
        List<P> ps = new ArrayList<>(); for (int i = 0; i < 2000; i++) ps.add(new P(1 + rnd.nextInt(5), i, false));
        // same-severity FIFO
        for (int i = 0; i < ps.size(); i++) for (int j = i + 1; j < Math.min(ps.size(), i + 50); j++) if (ps.get(i).severity == ps.get(j).severity && c.compare(ps.get(i), ps.get(j)) >= 0) fifo = false;
        // transitivity on random triples
```

```
        for (int k = 0; k < 200000; k++) { P a = ps.get(rnd.nextInt(2000)), b = ps.get(rnd.nextInt(2000)), d = ps.get(rnd.nextInt(2000)); if (c.compare(a, b) < 0 && c.compare(b, d) < 0 && c.compare(a, d) >= 0) trans = false; }
        System.out.println("  Equal-severity patients always served in arrival order : " + fifo);
        System.out.println("  Comparator transitive on 200,000 random triples        : " + trans);
        System.out.println("  Worst-case extra wait of a severity-5 patient vs a later severity-1 arrival: (5-1) x W = " + (4 * W) + " ticks");
        if (sink == 1) System.out.println();
    }
}
```

Output

```
Scenario: n minor (severity 5) patients waiting at tick 0; then every tick one critical (severity 1) patient arrives and the doctor treats one patient.
Run for 3000 ticks. 'first old served' = tick at which the first of the waiting severity-5 patients is treated (none = starved).
n waiting Design                    time (ms) patients scanned first old served old treated crit. treated crit. waiting at end
1000      strict priority (orig.)           1.9             0   none (starved)          0         3000           0
1000      re-scan + rebuild (fix 1)        25.7        300300             800       1000         2000         1000
1000      static aging key (redesign)       1.4             0             800       1000         2000         1000
10000     strict priority (orig.)           0.4             0   none (starved)          0         3000           0
10000     re-scan + rebuild (fix 1)        83.2       3000300             800       2201          799         2201
10000     static aging key (redesign)       1.3             0             800       2201          799         2201
100000    strict priority (orig.)           0.3             0   none (starved)          0         3000           0
100000    re-scan + rebuild (fix 1)       468.1      30000300             800       2201          799         2201
100000    static aging key (redesign)       1.0             0             800       2201          799         2201

Validation of the redesign (comparator properties and fairness):
  Equal-severity patients always served in arrival order : true
  Comparator transitive on 200,000 random triples        : true
  Worst-case extra wait of a severity-5 patient vs a later severity-1 arrival: (5-1) x W = 800 ticks
```

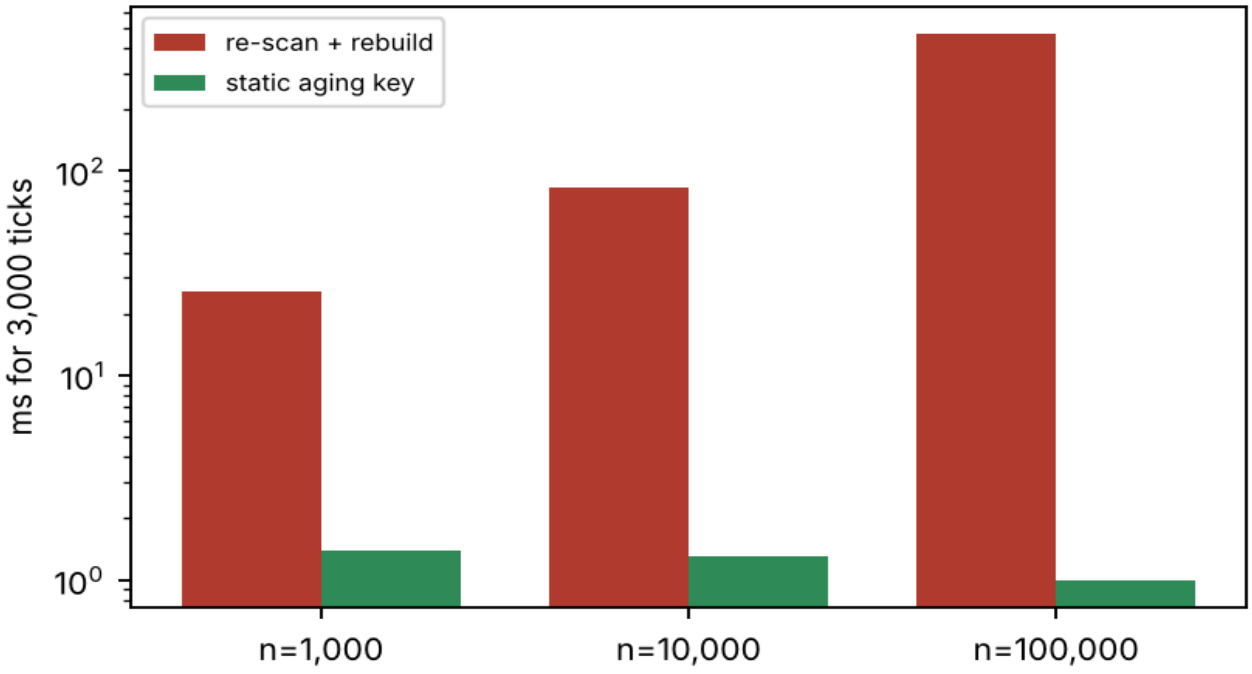


Figure 5 – Time for 3,000 ticks: re-scan + rebuild versus static aging key (log scale).

n waiting	Fix 1 (ms)	Redesign (ms)	Speed-up	Scanned (fix 1 / redesign)	First minor patient served
1,000	25.7	1.4	≈ 18×	300,300 / 0	tick 800 in both
10,000	83.2	1.3	≈ 64×	3,000,300 / 0	tick 800 in both
100,000	468.1	1.0	≈ 470×	30,000,300 / 0	tick 800 in both

F. Defence of the final design (2 marks)

- **Was the prediction supported?** Yes on all three points. Both designs treated the first minor patient at tick 800 and the same number of minor patients (1,000 / 2,201 / 2,201); the redesign scanned 0 patients; and the speed-up grew with n (≈ 18× → 64× → 470×).
- **Magnitude:** 468 ms → 1.0 ms at n = 100,000, and the work is independent of the number of waiting patients except for the O(log n) heap cost. At n = 1,000, the 25.7 ms of fix 1 is also negligible in absolute terms, so for small waiting rooms the original fix 1 would be acceptable.
- **Starvation removed:** the strict design never served a minor patient ("none (starved)"); the redesign bounds the extra wait to $(5-1)\cdot W = 800$ ticks.
- **New costs and trade-offs found by the experiment:** at n = 10,000 and 100,000, once the aged minor patients reach the head at tick 800 they are served one per tick for 2,201 ticks, and the newly arriving critical patients **wait** (2,201 critical patients still waiting at the end). Aging therefore does not create capacity: with a backlog far larger than W it just changes who waits. Choosing W is a clinical policy decision; a hospital might cap the boost so that a severity-5 patient can never outrank a severity-1 or 2 patient, and handle the backlog with more staff instead. The redesign also depends on the key formula, so the comparator should be unit-tested like Q5's.
- **When the original may still be preferable:** if arrivals never exceed capacity or a minor patient is never at risk (starvation cannot occur), strict priority is simpler and exactly predictable, and for

life-critical wards strict priority is a deliberate policy. Fix 1 is acceptable for small n (25 ms for 1,000 waiting patients).

Conclusion: the improvement came from understanding the problem (uniform aging preserves order, so the key can be fixed at admission), not from swapping one collection for another: the same PriorityQueue class is used before and after, with a different key.